

NSS Reflection Report

Benjamin Kofi Yankey

AmaliTech : Independent Training & Practice Portfolio

Period Covered: December 1, 2025 -- June 2026 (~7 months)

33 independently attributable training/practice repositories

(37 project folders reviewed in total, including 4 no-history/empty folders excluded from evidentiary claims)

Part 1 — Plain-Language Narrative

Overview

This report covers 33 independently attributable training and practice repositories built between December 1, 2025 and June 2026 (~7 months), ranging from basic shell-scripting exercises to solo capstone-style AWS builds. Of those 33, one (DataPulse_Team8) was a genuine multi-person team project; the rest were solo, self-directed work, and I've kept that distinction explicit throughout rather than blur it.

Stage 1: Foundations (December 2025 -- February 2026)

- git-lab — a genuine, executed Git workflow exercise (real branch, real fast-forward merge, real reflog inspection, not just narrated).
- devops_challenge — a sysadmin "starter pack" script verified on re-test to still have an unresolved idempotency bug and a broken generated backup script (discussed honestly below).
- bashScripting and Data-Structures-and-Algorithms — core scripting and algorithm fundamentals, including a real self-caught quicksort mislabeling.
- aws-automation-project — Bash/AWS CLI tooling that grew into a hand-rolled, S3-backed state tracker.
- 3tier-project — a full Terraform 3-tier AWS architecture, with a genuine security-hardening pass and, on closer inspection, some unresolved security debt (see caveats).
- terraform-aws-infrastructure and IaC — two smaller, cleanly-verified single/few-commit AWS foundations exercises.
- OpenStack — despite the name, a small serverless API Gateway/Lambda demo.
- terraform-ansible-webapp — Terraform + Ansible provisioning a task-manager app, with real if partly-unwired security tooling (firewalld/SELinux/auditd hardening plus a threat-detection script and OPA policies that are never actually invoked).

- `yaml-config-lab` — a YAML/config-management exercise whose more advanced pieces turned out, on direct execution, not to actually work (detailed honestly below).
- `app` — a small Express/MySQL contact-form app, also found on re-inspection to have a committed plaintext database password and an XSS gap.
- `serverless-task-management` (late Jan–Feb 2026) — a genuine full-stack serverless app (React/TypeScript on Amplify, six Lambda functions, Cognito auth with Google/GitHub OAuth, DynamoDB) where a late security pass fixed a real broken-access-control bug letting any authenticated user list all accounts, replaced full-table database scans with indexed queries, and disabled an insecure OAuth flow; the admin dashboard's data views are real, but several action buttons only update local browser state and don't call a backend at all.
- `docker-lab` and `flask-backend` — container security fundamentals, including a same-day catch-and-fix of a committed SSL private key in `flask-backend`.
- `docker-ec2-project` — a two-tier Flask/MySQL app taken to a GitHub Actions pipeline with Trivy scanning; AWS Secrets Manager re-verification found the "automated EC2 deploy" is currently only exercised via manual trigger, not on every push.
- `Project_Complete_CICD_Pipeline_Jenkins` — a Jenkins CI/CD + Terraform project where a teammate, Yram Tetteh-Abotsi, laid down an initial 3-commit skeleton and I did essentially all of the substantive build-out (28 of 31 commits): parallel security scanning, multi-platform Docker builds, a dedicated Secrets Manager/IAM/VPC-endpoints hardening pass, and two concrete bug fixes; a committed SSH private key and Terraform state file in this repo are worth flagging alongside that hardening work.
- `commentary-blog` — a full-stack React/Express blog built in a compressed 3-day, 103-commit sprint, with two genuinely different, working CI/CD pipelines; re-verification also caught that the project's own sprint-retrospective document is internally inconsistent and describes tests/coverage/features that don't match the actual code.

Stage 2 — The One Real Team Project: DataPulse (March 2026)

- `DataPulse` (solo scaffold) — a solo FastAPI scaffold built in one day: genuinely complete on the auth and file-upload paths, with a real, verified bcrypt UTF-8 truncation bug fixed properly, but with the core "quality scoring" business logic left as an honest, explicit stub.
- `DataPulse_Team8` — joined days later: a real multi-person team build of the same idea (~280 total commits, at least 9 other contributors, my share 111 commits/~40%) under a genuine ~12-day deadline.
- Re-verified authorship line by line rather than trusting git log --author: several items previously credited as my own feature work (a Prometheus metrics endpoint, a production data-

generator script, frontend pagination, and a substantial test-coverage PR) turned out to be **teammates' code that I merged as the repo's maintainer** — now reported only as integration work, not authored code.

- What remains solidly mine, re-confirmed against the current passing Terraform/CI configuration: the DevOps/platform track — the Terraform AWS deployment, CodeDeploy blue-green hardening, the Prometheus/Grafana/Loki monitoring pipes, and a genuine migration of CI authentication from static AWS credentials to GitHub OIDC.

Stage 3 — Independent, Higher-Complexity Capstones (June 2026)

- Poly-Orchestrator ("ShopNow") — a full e-commerce storefront (cart/checkout, reviews with moderation, address management, a role-gated employee console, Google sign-in) built to compare AWS ECS Fargate against EKS as deployment targets for the same app.
- Re-verification confirmed the customer-facing features are genuinely working (real Postgres/Redis-backed endpoints, server-side-enforced RBAC on the employee console, not just a UI mock).
- In its final week the work shifted entirely to infrastructure: replacing a self-managed Jenkins pipeline with a fully Terraform-defined AWS CodePipeline + CodeBuild setup, personally debugging two real failures that only surfaced once the pipeline went live (a shell-dialect bug and a Docker Hub rate limit), and patching an OpenSSL CVE across all three service images the same day as the cutover.
- Caveat: there is no automated test suite anywhere in this repo, for either the frontend or the backend services.
- Correction on re-verification: an earlier pass had read this as 14-of-18 commits with implied teammates; re-checking the full, unfiltered git history across all three of my git identities confirmed it is **entirely solo, 19 of 19 commits**.
- FinOps — a simulated wasteful AWS account plus real cleanup/governance automation (EBS garbage collection, tag enforcement, cost analysis).
- Artifacts — a single-sitting build of a secure software supply chain and a cross-region disaster-recovery plan, re-verified as a genuinely complete, internally consistent design, though unproven against a real AWS run.
- AI-Devops/TechStream — a self-healing infrastructure demo; re-verification confirmed this is a **real, working detect→alert→remediate loop**, not a superficial mockup (Prometheus recording rules genuinely feed Alertmanager, which genuinely triggers a Docker-API restart through an allowlisted, cooldown-gated "healer" service, mirrored in a parallel AWS/Lambda path).

- These three (FinOps, Artifacts, AI-Devops/TechStream) each landed as one or two dense commits rather than an iterated project, so sustained ownership can't be claimed from any one individually — but together, alongside Poly-Orchestrator, they show the technical range (security supply chains, disaster recovery, cost governance, orchestrator comparison, SRE/self-healing systems) applied by the end of this period.

Growth in Responsibility, Stated Honestly

- A visible, continuous trajectory from basic scripting/algorithms (Dec 2025) through infrastructure practice, into a first real team project (Mar 2026), into increasingly ambitious, higher-complexity independent capstone work (Jun 2026).
- Recurring, re-verified evidence of catching and fixing real defects: the destructive `rm -rf` in `devops_challenge`, the public-internet-exposed database in `3tier-project`, the committed SSL key in `flask-backend`, the `bcrypt` truncation bug in `DataPulse`, and the AWS EC2 user-data limit in `AI-Devops`.
- One real instance of team-embedded work (`DataPulse_Team8`), correctly re-scoped this pass to exclude commits that were actually teammates' authored code merged under my name.

Business Impact, Stated Honestly

Almost all of this portfolio consists of self-directed practice repos with no external users or production stakeholders, and should not be read as business impact. The one exception is `DataPulse_Team8`: my re-verified contribution there was keeping a 9+ person team's CI/CD and Terraform deployment pipeline working under a real ~12-day deadline, including a genuine security improvement (migrating CI auth from static credentials to OIDC). Everywhere else, the honest framing is skill demonstration and independent capability-building, not measurable business outcomes.

Honesty Notes and Caveats

Compiled from this pass's code-level re-verification, not just commit messages — kept in deliberately rather than smoothed over.

- **3tier-project**: current code genuinely implements RDS Multi-AZ/encryption, a closed database security group, and Secrets Manager-backed passwords — but a full Terraform state backup and an SSH private key were committed to git history early on and, although later removed from HEAD, remain permanently recoverable via `git show`. Separately, the bastion's SSH lockdown mechanism exists in code but `admin_cidr` still defaults to `0.0.0.0/0` and is never overridden, so SSH is open to the internet as actually configured, and an SSM-related inbound rule on the app tier reopens a public port the rest of the hardening was trying to close.

- **app:** re-inspection found a real-looking AWS RDS hostname and a plaintext password committed in .env since the first commit (permanently in git history even if since rotated), plus unescaped user input rendered into HTML (stored/reflected XSS risk) — neither was caught in the original pass.
- **docker-ec2-project:** the CI/CD "automated deploy" only fires when the workflow is run manually with specific inputs; on a normal push it is silently skipped due to how the credential-conditional bugfixes were ultimately resolved. The Trivy vulnerability scan step also has a trailing || true, so it can never actually fail the build regardless of findings.
- **DataPulse_Team8:** several commits I originally credited to myself (a metrics endpoint, a data generator, frontend pagination, a test-coverage PR) are GitHub "merge pull request" commits where the underlying code was authored by teammates — corrected in this report to be credited as integration/maintainer work, not authored feature work.
- **commentary-blog:** the project's own FINAL_RETROSPECTIVE.md describes a 12-day/42-commit timeline with 45 tests and 87% coverage, none of which matches the real git history (103 commits across 3 days) or the real test suite (~13 test cases, near-zero frontend coverage). Reported as a documentation/template-honesty issue, not a reflection on the actual (genuinely functional) code.
- **FinOps:** a commit titled "Refactor code structure for improved readability and maintainability" was confirmed, on diffing, to contain only new documentation files and zero source-code changes.
- **Poly-Orchestrator:** an earlier verification pass misread this as "14 of 18 commits, majority contributor," implying teammates. Re-checked against the full unfiltered history across all three of my own git identities and corrected to entirely solo (19/19). Separately, its "Refactor code structure" commit is also mislabeled — it's actually 6,400+ insertions of new feature work, not a refactor — and Google sign-in, while fully implemented, ships feature-flagged off (no client ID configured) rather than active.
- **Project_Complete_CICD_Pipeline_Jenkins:** a real SSH private key and Terraform state files are committed to the repo, sitting oddly next to the security-hardening work done elsewhere in the same pipeline.
- **yaml-config-lab:** the README's "Security Improvements" claims don't hold up on execution — a sibling JSON config file still has the plaintext password the YAML file had removed, several validation/test scripts are byte-corrupted (literal n/t instead of real line breaks) and silently no-op instead of running, and the demo Flask app actually crashes on startup because envsubst doesn't support the \${VAR:-default} syntax the config relies on — all confirmed by direct execution, not just reading the code.

- **terraform-ansible-webapp:** a real threat-detection script and a set of OPA/Rego security policies both exist and work standalone, but neither is actually wired into the deployment (no cron/systemd scheduling for the script, nothing in the repo invokes OPA against the policies) — real security code that isn't part of the running system.
- **serverless-task-management:** the admin dashboard's user-list and task-stats views are genuinely backend-wired, but several of its action buttons (add/delete user, broadcast email, system settings) only mutate local browser state and log a fake activity entry — they don't call any backend endpoint.
- **IaC:** the README's "Author" byline names a different person than the git history shows; both commits are unambiguously mine, so this reads as stale template boilerplate carried over from a shared training template rather than a false authorship claim — but it's a real inaccuracy that should be corrected at the source.
- **docker-lab:** a more polished-looking README, changelog, contributing guide, and CI workflow exist in the working folder but were never actually committed to git — only the original, shorter, committed version should be credited as tracked work.
- **No git history, but real content:** try, terraform, and Containerized all show evidence of an actually-applied Terraform deployment (state files with real, live resource data) despite never being committed to version control, so they're reported as verified content without a timeline rather than excluded outright; Orchestrator is a complete, coherent ECS-vs-Kubernetes design with no evidence it was ever actually run.
- **Excluded from all claims entirely:** 0-terraform and docker (trivial, no git history, no evidence of deployment), sms-node-project and Hands-on Artifacts (empty/unimplemented scaffolds — every implementation file in sms-node-project is confirmed 0 bytes), diveintoansible-lab (empty folder), and desksec-server (zero commits under my identity — all four belong to a teammate).

Summary

Over roughly seven months, I moved from basic shell-scripting and algorithm practice, through increasingly ambitious solo infrastructure builds, into one real team project, into independent, higher-complexity capstone work by the end of the period. Re-verifying the underlying code rather than trusting commit messages alone surfaced both stronger confirmation of some claims (the AI-Devops self-healing loop, the Poly-Orchestrator sole-authorship correction) and a set of honest corrections (committed secrets still in git history, a misattributed test-coverage PR, gaps between documentation and reality) that I think are more useful to an advisor named plainly than smoothed over.

Milestones — Quarter Highlights

The three milestones below are pulled from the repositories detailed in Part 2 and re-stated in a milestone-report format: context, what was actually done, and the verified impact.

Migrated DataPulse_Team8's CI/CD authentication from static credentials to GitHub OIDC and hardened the blue-green deploy pipeline	
Context	DataPulse_Team8 was a real 9+ person team build under a ~12-day deadline. The CI/CD pipeline was authenticating to AWS with long-lived static credentials, and the CodeDeploy blue-green rollout had no guard against overlapping deployments.
What you did	• Replaced static AWS credentials with a scoped GitHub OIDC provider and IAM roles restricted to the org repo • Added stop-active-deployment and concurrency guards to the live CodeDeploy blue-green workflow • Authored the DevOps documentation (README + DEPLOYMENT.md, ~2,800 lines combined) so the rest of the team could operate the pipeline independently
Impact	Removed long-lived AWS credentials from CI entirely (a genuine security improvement, not just a config change), and kept a 9+ person team's deployment pipeline working under a hard ~12-day deadline.

Closed a broken-access-control vulnerability and removed an insecure OAuth flow in serverless-task-management	
Context	The admin panel's list-users function had no admin check at all — any authenticated user could enumerate every account in the Cognito pool. Separately, the task-listing function relied on unbounded full-table scans, and the Cognito configuration still allowed an insecure OAuth "implicit" flow.
What you did	• Added a proper admin-group check to the list-users function, closing the access-control gap • Replaced the unbounded Scan with an indexed Query (admin path) and batched BatchGet calls (member path), eliminating every Scan call in the application • Disabled the OAuth implicit flow in the Cognito Terraform config, restricting sign-in to the code flow only
Impact	Removed a real account-enumeration vulnerability and a legacy OAuth weak point from a production-shaped app, and cut the admin dashboard's worst-case database access pattern from a full-table scan to an indexed lookup.

Migrated Poly-Orchestrator's CI/CD from a self-managed Jenkins pipeline to a fully Terraform-defined AWS CodePipeline + CodeBuild setup	
---	--

Context	Poly-Orchestrator ("ShopNow") was a solo-built e-commerce storefront comparing ECS Fargate against EKS as deployment targets. In its final week, the remaining work was entirely infrastructure: retiring the self-managed Jenkins setup in favor of an IaC-defined pipeline.
What you did	• Defined the full pipeline (CodePipeline, CodeBuild, scoped IAM roles for ECR/ECS/EKS/S3) in Terraform behind an `enable_cicd` variable • Diagnosed and fixed a POSIX-sh substring bug that only surfaced once the pipeline went live (CodeBuild's shell is dash, not bash) • Diagnosed and fixed a Docker Hub rate-limit failure by re-pulling the base image through ECR Public • Patched an OpenSSL CVE across all three service Dockerfiles the same day as the cutover
Impact	A working, IaC-defined CI/CD pipeline replaced a self-managed one, with two live-only failures found and fixed the same week they appeared — and the security patch shipped the same day as the migration rather than being deferred.

Notes for your mentor

- Biggest learning this period: re-verifying against the actual code (not just commit messages) surfaced both stronger confirmation of some claims and a few corrections I'd rather report plainly than smooth over (see Honesty Notes and Caveats above)
- Next period I want to close the gaps this pass surfaced — particularly the still-open `admin_cidr` default in 3tier-project and the missing automated test suites in Poly-Orchestrator and serverless-task-management
- I'd value feedback on whether the DataPulse_Team8 re-scoping (removing merge-commit credit that was actually teammates' authored code) is the right level of caution, or overcorrected

A. Team-Based Project — DataPulse_Team8	
GitHub	https://github.com/Benjamin-yankey/DataPulse_Team8
Timeframe	2026-03-09 to 2026-03-13 — 111 of 280 total team commits (~40%, re-confirmed via two independent author-matching methods), within a ~12-day project window. At least 9 other contributors plus a bot account.
Stack	FastAPI/SQLAlchemy, PostgreSQL 15, Pandas, pytest, Nginx-served HTML/JS frontend, Docker/Compose, Terraform (ECS/Fargate, ECR, RDS, ElastiCache, ALB, CodeDeploy blue-green), GitHub Actions with OIDC, Prometheus/Grafana/Loki.

Features (own work, re-verified)	• 3a8084e — GitHub OIDC for credential-less AWS authentication in CI, replacing static credentials. • db410ee, 70ed4c4, 313cb57 — Terraform for EC2/ECS/RDS/ElastiCache and ECS/ECR blue-green deploy config; current terraform/ (12 files, ~3,500 lines) passes terraform validate cleanly. • 81ea7d8 — appspec.yaml for AWS CodeDeploy blue/green ECS deployment.
Bug Fixes	• 86a68e3 — fixed CI workflow paths after directory restructuring; 8bc28f3 — corrected YAML indentation breaking the CI build. • 06bf95c/851d372 — fixed CORS, database connection, and security-group misconfigurations in the deployed environment. • 982a67c — security-vulnerability fix: hardening across Dockerfiles/compose configs.
Infra / CI-CD (verified against current repo state)	• OIDC provider + scoped IAM roles restricted to the org repo (terraform/github_oidc.tf), confirmed present and functioning in deploy-ecs.yml (role-to-assume) — a complete, real migration, not left half-done. • CodeDeploy blue-green hardening (stop-active-deployment step, concurrency guards) confirmed present in the live deploy-ecs.yml workflow. • Prometheus/Grafana/Loki monitoring: confirmed functional data pipes (correct scrape targets, working Loki push URL) but no committed Grafana dashboard JSON or Prometheus alert rules — "monitoring" here means working collection, not curated dashboards/alerts.
Docs	• a5201a1 — sole-authored devops/README.md, verified as a single 1,843-line addition. • 39877d6 — DEPLOYMENT.md (984 lines).
Notable / High-Impact Contributions	• End-to-end AWS deployment pipeline including a real security migration — built and hardened a genuinely complete Terraform + GitHub Actions + CodeDeploy pipeline and migrated CI auth to scoped OIDC roles, confirmed against the live repo. • Sole author of the DevOps documentation (1,843 + 984 lines) — substantial, verifiable written contribution. • Iterative production troubleshooting under time pressure across the whole window.
Important Correction From Re-Verification	Several commits previously credited to me — a Prometheus /metrics endpoint (PR #55), a production data-generator script (PR #41), frontend pagination/filtering (PR #68), a UI redesign (PR #77), a test-coverage PR (PR #59), and project-tooling PR #48 — are two-parent merge commits where the underlying branch was authored by teammates (gisacaleb72@gmail.com, tijanimubarak@ChefIT.local). GitHub attributes the merge commit to whoever clicks "merge," which inflated my apparent feature/testing output in earlier passes. These are now reported only as integration/maintainer work, not authored code. By contrast, self-merges of my own branches (313cb57, bd0b045, da380ac, 370eb72, bfe63be) were individually verified as authored by me and are credited normally.

B.1 FinOps (Cost-Detective-Audit)

GitHub	https://github.com/Benjamin-yankey/FinOps
Timeframe	2026-05-28 to 2026-06-01, 4 commits, solo (re-confirmed count via full unfiltered git log; earlier drafts said 3).
Features (verified as real, working automation)	• 1035589 — initial buildout: wasteful-resources Terraform module, AWS Budgets, AWS Config governance with a Tag Enforcer Lambda, Spot-based ASG reference architecture, EBS garbage collector. • <code>1035589</code> — <code>ebs_garbage_collector.py</code> — confirmed real: dry-run by default, skips tagged/young volumes, supports snapshot-before-delete, estimates cost, writes a report. • <code>1035589</code> — <code>tag_enforcer.py</code> (Lambda) — confirmed real: parses Config/EventBridge events, re-verifies tags via <code>describe_instances</code> to handle evaluation lag, then stops or logs per an ACTION env var. • <code>1035589</code> — <code>ac16f01</code> — <code>nat_gateway_analyzer.py</code>, confirmed functional (real CloudWatch queries, real cost math), though it lacks a CLI and hardcodes region/threshold defaults. • <code>3d1a892</code> — <code>destroy_all.sh</code>: confirmed genuine operational safety — dry-run (plan -destroy) by default, requires typing "destroy" at an interactive prompt unless --yes is passed.
Verified Stub	<code>terraform/modules/scheduler/main.tf</code> contains only a variable and one CloudWatch event rule, with an explicit in-code comment: "NOTE: This is a placeholder for the full module." No Lambda, IAM role, or target is actually defined.
Mislabeled Commit	Commit <code>be06305</code> , titled "Refactor code structure for improved readability and maintainability," was confirmed via <code>git diff --stat</code> to contain 1,032 insertions across 14 Markdown/screenshot files and zero source-code changes.
Notable / High-Impact Contributions	• <code>ebs_garbage_collector.py</code> and <code>tag_enforcer.py</code> are the two most solid pieces of work — complete, safety-conscious automation, not throwaway scripts. • <code>destroy_all.sh</code>'s interactive confirmation gate and pre-flight validation reflect real operational-safety thinking. • The scheduler module's own honesty (flagging itself as a placeholder rather than presenting incomplete work as finished) is itself worth noting.

B.2 Artifacts

GitHub	https://github.com/Benjamin-yankey/Artifacts
Timeframe	2026-06-22, 1 commit, 68 files, 3,765 lines (confirmed via <code>git show --stat</code>).

Features (re-verified against the actual code, not just the README)	• Terraform network/RDS/ECR/backup/cicd modules — dual-provider design (primary + DR region alias), all cross-module variable/output references confirmed to resolve correctly. • pipeline/scan-gate.sh — confirmed a real, functioning gate: polls ECR scan findings for up to ~400s and exits 1 on any HIGH/CRITICAL finding, genuinely wired into buildspec.yml's post_build phase. • DR scripts (dr-create-ondemand-backup.sh, dr-restore.sh, dr-simulate-failure.sh, teardown.sh) — a coherent, real automation chain (backup → delete → restore → readiness wait → elapsed-time reporting against a 30-minute target), not a diagram. • A small but real FastAPI double-entry ledger app with an Alembic migration and a genuine pytest suite (transfer invariants, insufficient-funds and self-transfer rejection).
Minor Rough Edges Found on Inspection	• Dead/vestigial code in dr-create-ondemand-backup.sh (a discarded terraform output call joined with ; instead of &&). • An unused DR_KMS_KEY_ARN variable never referenced by the restore script. • A runbook referencing a Terraform output (backup_plan_id) that doesn't exist in outputs.tf.
Notable / High-Impact Contributions	• The vulnerability gate is genuinely load-bearing, not decorative — it is actually invoked from the pipeline and actually fails the build on findings. • The DR restore path is coherent, real automation with a runbook that hedges its timing claim ("usually 8--15 minutes") rather than overpromising.
Caveat	Single commit, no Terraform state, no CI run output, and no captured DR-drill log anywhere in the repo — every "works"/"under 30 minutes" claim is a design/code-level claim, not an empirically demonstrated one.

B.3 AI-Devops (TechStream)

GitHub	https://github.com/Benjamin-yankey/AI-Devops
Timeframe	2026-06-23 to 2026-06-24, 2 commits (100% of repo history).

Overview	• Features — re-verified as a real, working closed loop, not window dressing • Prometheus recording/alert rules define real ratios (error rate, p95 latency, queue depth, cache-hit ratio) carrying a remediation_target label; Alertmanager routes critical alerts to a webhook; healer.py receives it and restarts the named Docker container, gated by an explicit allowlist and a 60-second cooldown, with every action logged to a JSONL audit file. • The chaos-injection tooling actually hits real /chaos/* endpoints that inject real HTTP 500s, latency, slowed DB queries, a disabled cache, CPU burn, and a stalled worker — each mapped to the specific alert it's meant to trigger. • The AWS path mirrors this: CloudWatch alarm → EventBridge → a Lambda that issues an SSM restart command and scales the ASG out, both logged. • The "AI root-cause analysis" script is a real, modest rule-based correlation engine (baseline-vs-peak amplification, tier-scoped candidates, cross-referenced against the healer's action log) that only calls out to Claude if an API key is present — no hardcoded key, no fabricated LLM call.
Bug Fixes	• 26dae39 — fixed an EC2 boot failure by enlarging the root EBS volume (gp3, 30GB) after the default volume proved too small for Docker + app images + the CloudWatch agent. • 26dae39 — fixed a genuine AWS platform constraint: the rendered user-data script (~41KB) exceeded AWS's 16KB limit, fixed by switching from base64encode to base64gzip. • 26dae39 — fixed a fragile teardown script that could pick up a corrupted terraform output value; added sanitization with a safe fallback.
Honest Weaknesses Found	• docker-compose.yml hardcodes demo credentials (Postgres/Grafana admin passwords) — low-risk for a disposable local demo but worth flagging. • No automated test suite — validation is via chaos scripts and manual observation. • The healer service depends on Docker-socket access, a real production security concern the demo doesn't address (acceptable for a training project, relevant if ever framed as production-ready).
Notable / High-Impact Contributions	• The AWS user-data fix is the strongest evidence of engineering judgment — a genuine platform constraint, correctly diagnosed and fixed. • The detect→alert→remediate loop is real engineering, independently traced end-to-end through the Flask app, Prometheus rules, Alertmanager config, and the healer service — confirmed working, not just described.

B.4 Poly-Orchestrator ("ShopNow")	
GitHub	https://github.com/Benjamin-yankey/Poly-Orchestrator
Timeframe	2026-06-02 to 2026-06-22, 19 of 19 commits — corrected this pass to entirely solo (see below).

Stack	Angular 18 SPA, two Node/Express microservices (Products on Postgres, Cart on Redis), Docker multi-stage builds, Terraform (VPC/ECR/ECS Fargate/EKS), AWS CodePipeline + CodeBuild, JWT auth with Google Identity Services.
Overview	• Features — verified against the current codebase, not just commit messages • Cart & checkout: a real per-user Redis-backed cart (JWT-authenticated) plus wishlist/save-for-later; real order rows created through the Products service's order endpoints. • Product reviews with moderation: a real Postgres reviews table, public submit/list endpoints, and admin approve/reject endpoints, matched by a frontend review UI with an "awaiting moderation" state. • Employee dashboard: a ~1,650-line tabbed console (orders, inventory, support, returns, coupons, reviews) gated per-tab by a department-capability model enforced server-side, not just hidden in the UI — a legitimate RBAC design. • Google sign-in: a real, complete implementation (Google Identity Services, ID-token verification against Google's tokeninfo endpoint) — but feature-flagged off by default, since no client ID is configured in this repo's checked-in config, worth stating precisely rather than implying it's live.
Bug Fixes	• 45c3079 — fixed a POSIX-sh substring incompatibility in the CodeBuild buildspec (CodeBuild's shell is dash, not bash). • 8243b18 — fixed a Docker Hub rate-limit failure by pulling the base image from ECR Public and re-tagging it locally before the build.
Infra / CI-CD — verified against actual Terraform/buildspec	A complete migration from a self-managed Jenkins pipeline to AWS CodePipeline + CodeBuild, fully defined in Terraform (real aws_codepipeline, aws_codebuild_project, scoped IAM roles for ECR/ECS/EKS/S3), gated by an enable_cicd variable — confirmed a complete, working pipeline definition, not a stub. An OpenSSL CVE patch (apk upgrade --no-cache) was applied across all three service Dockerfiles the same day as the cutover — a real fix, though a rolling upgrade rather than a pinned version.
Testing	Confirmed: no automated tests exist anywhere in the repo, for any of the four codebases (frontend, cart, products, or the frontend's Express proxy) — worth stating plainly rather than omitting.
Notable / High-Impact Contributions	• The CI/CD platform migration — a complete, IaC-defined pipeline cutover, followed by personally debugging two real failures that only surfaced once it went live. • Server-side-enforced RBAC and moderation — confirmed by reading the actual auth/capability code, not assumed from the commit message.
Correction From Re-Verification	An earlier pass read this repo as "14 of 18 commits," implying a majority-but-not-sole contribution with teammates. Re-running git log across the full, unfiltered history and all three of my own git identities (work email, personal Gmail, and the GitHub-auto-attributed merge-commit identity) found no second contributor anywhere in the history and only one remote, the user's own fork — corrected to entirely solo, 19 of 19 commits.

B.5 serverless-task-management	
GitHub	https://github.com/Benjamin-yankey/serverless-task-management
Timeframe	2026-01-29 to 2026-02-19, 14 substantive commits, solo.
Stack	React/TypeScript on AWS Amplify, AWS Lambda (Node.js), API Gateway with Cognito authorizers, DynamoDB, Cognito (incl. Google/GitHub OAuth), SNS notifications, Terraform.
Features	A working task-management app for an organization (signup restricted to company email domains via a Cognito pre-signup Lambda), with Admin and Member roles, six backend Lambda functions, and the full AWS infrastructure defined in Terraform — delivered mostly in one large initial commit, followed by a second phase adding a built-out admin dashboard and Google/GitHub social sign-in through Cognito.
Security Fixes — all verified directly against current code	• Fixed a real broken-access-control bug: the list-users function originally had no admin check at all — any authenticated user could enumerate every user in the Cognito pool. Confirmed fixed with a proper admin-group check. • Replaced full-table database scans with indexed queries: the task-listing function previously ran an unbounded Scan for the admin view; confirmed now using a Query against a dedicated GSI (admin path) and batched BatchGet calls (member path) — no Scan calls remain anywhere in application code. • Disabled an insecure OAuth "implicit" flow, confirmed in the current Cognito Terraform config (code-flow only), and removed an unneeded IAM permission from a role that no longer needed it.
What's Real vs. What's a UI-Only Mock	The Google/GitHub OAuth wiring is real end-to-end (Terraform-provisioned identity providers, a real Amplify redirect call). The admin dashboard's user list and task stats are genuinely backend-wired. But its "Add User," "Delete User," "Broadcast Email," and "System Config" actions only mutate local React state and log a fake activity entry — none of them call a backend endpoint, which should be described as a working read-only admin view with several UI-only mock actions, not a fully functional admin backend.
Testing	No automated test suite exists anywhere in the repo (confirmed via a repo-wide search for test frameworks). A 1,268-line TESTING.md is a manual QA checklist, not automated test code.
Notable / High-Impact Contributions	The security-hardening pass (broken access control, scan-to-query rewrite, OAuth flow fix) is the strongest evidence of engineering judgment in the repo — all three fixes independently re-verified against current code, not just commit messages.
Caveat	~5,400 lines landed in one initial commit with no prior history, so it isn't possible to tell from git alone how much was hand-written incrementally versus assembled in one sitting — described here as "assembled the initial full-stack architecture," not implying line-by-line manual authorship of every file.

C.1 commentary-blog (DevLog)	
GitHub	https://github.com/Benjamin-yankey/commentary-blog

Timeframe	2026-02-16 to 2026-02-18, 103 commits (59/41/3 across the three days), solo.
Features — re-verified as genuinely implemented, not stubs	• JWT register/login with real bcrypt hashing and real validation; post CRUD with real pagination (LIMIT/OFFSET + count query); comments with real FK-constrained persistence and a fully wired React component; author-only edit/delete enforced server-side (403 on ownership mismatch) with a client-side convenience layer on top — correct enforcement order. • The Postgres → SQLite → Postgres migration is real and still visible in the current tree: both config files exist, with Postgres wired in as the active one.
Confirmed Not Real	Like counts and "47 Comments" in the post-view footer are fake: likeCount is a random number set once on load and never persisted, and the comment count is a hardcoded literal ignoring the real fetched count — this should not be described as a working engagement feature.
CI/CD — both pipelines read directly	• GitHub Actions: real Postgres service container, ESLint, Jest with coverage, Codecov upload — but the lint step ends in <code> true</code>, so it can never actually fail the build (advisory only). • Jenkins: a materially different, fuller pipeline (parallel lint/test, Docker image build, frontend production build) whose lint step can genuinely fail the build — but its "Coverage Check" stage just greps the coverage HTML for the string "All files" and echoes a fallback if not found; its own comment admits there's no real enforced threshold.
Documentation Honesty Issue — Independently Re-Verified	docs/FINAL_RETROSPECTIVE.md claims a 12-day, 42-commit project with 45 tests and 87% coverage. None of this matches reality: actual git history is 103 commits across 3 calendar days, and the actual suite is ~13 test cases with near-zero frontend coverage on most files (confirmed via the committed lcov.info). The document is also internally inconsistent (elsewhere claims "15 total commits"), and its "Future Enhancements" section lists comments/editing/image-uploads as not-yet-built — despite all three being fully implemented in the same commit history. This reads as a course-template artifact filled with placeholder figures, not a record of actual delivery.
Notable / High-Impact Contributions	• A real, working core feature set delivered under compressed time — substantive full-stack work, not a shell. • Two genuinely different, functioning CI/CD pipelines, each with one identifiable but different weak spot (advisory lint vs. fake coverage gate). • A real, verifiable database migration under test coverage.

C.2 docker-ec2-project

GitHub	https://github.com/Benjamin-yankey/docker-ec2-project
Timeframe	2026-02-10 to 2026-02-26, 10 commits, solo.

Features	• ea862ab — AWS Secrets Manager integration for DB credentials with env-variable fallback (verified in web/app.py). • f7c528d — destroy.sh: confirmed genuine interactive safety checks before pruning Docker resources or deleting the AWS secret. • c96dc13 — GitHub Actions pipeline: build/test job plus a deploy-to-EC2 job over SSH.
Bug Fixes and Their Real Limits	• Credential-handling in CI was fixed across three commits, but the final if: conditions on the AWS-credentials and deploy steps check only manual-dispatch inputs — on a normal push, both steps are silently skipped, so "automated deploy" does not currently fire automatically. • The Trivy scan step ends in true, so it produces logs but can never fail the pipeline on findings.
Refactors / Tech Debt	web/app-with-secrets.py still exists in the tree alongside web/app.py but the Dockerfile only builds app.py — dead code left over from an incomplete cleanup. README also still links to three files deleted in the same restructuring commit.
Notable / High-Impact Contributions	• AWS Secrets Manager integration is genuine security-conscious engineering, though only partial: docker-compose.yml still ships plaintext default passwords, and the app's own fallback path defaults to the same hardcoded password. • The CI/CD debugging arc is an honest, realistic cycle — verified positively for the test job, verified with a real caveat for the deploy job's gating.

C.3 docker-lab	
GitHub	https://github.com/Benjamin-yankey/docker-lab
Timeframe	2026-02-10, 1 commit (100% of tracked repo history).
Features	A real, working (if minimal) Flask app; two genuinely different Dockerfiles (standard vs. a real multi-stage Dockerfile.prod using Gunicorn + a HEALTHCHECK); a substantive, specific 114-line SECURITY.md whose claims (non-root UID 1000, chmod 555, cap_drop: ALL, no-new-privileges) all check out against the actual docker-compose.yml/Makefile.
Important Finding: Uncommitted "Polish"	A GitHub Actions CI workflow, an expanded "production-ready" README, a CHANGELOG.md, and a CONTRIBUTING.md all exist in the working folder today but are untracked in git — never actually committed. The tracked, evaluable work is only the original single commit: the app, two Dockerfiles, compose file, Makefile, security doc, and a short, honest ~54-line README. The "production-ready" framing on disk should not be credited as part of the tracked contribution.
Notable / High-Impact Contributions	The Dockerfile vs. Dockerfile.prod split plus a specific (not generic) SECURITY.md is legitimate, verifiable evidence of container-security awareness within the actual tracked commit.

C.4 flask-backend	
GitHub	https://github.com/Benjamin-yankey/flask-backend
Timeframe	2026-02-10, 3 commits, solo, single day.
Features	A real Flask app behind an Nginx reverse proxy with load balancing, caching, rate limiting, and health checks, all in one large initial commit.
Security Fix	• 0777750 — added .gitignore and removed an already-committed SSL private key and Diffie-Hellman parameters file from tracking. • dda29ba — removed SSL/TLS support entirely (dropped the HTTPS block, HSTS header, port 8443), reverting to plain HTTP as a simplification following the key exposure.
Notable / High-Impact Contributions	Catching the committed private key is the strongest evidence of good security hygiene in this small sample.

C.5 Project_Complete_CICD_Pipeline_Jenkins	
GitHub	https://github.com/Benjamin-yankey/Project_Complete_CICD_Pipeline_Jenkins
Timeframe	2026-02-11 to 2026-02-27, 28 of 31 commits mine. A teammate, Yram Tetteh-Abotsi, authored the initial 3-commit skeleton (Feb 6--10: a basic app, a ~106-line Jenkinsfile, a first pass at Terraform) before I took the project over.
Stack	Node.js/Express, Jest+Supertest, Docker Buildx, Jenkins (Groovy declarative pipeline), Terraform (9 modules), AWS (EC2/Secrets Manager/IAM/CloudWatch), Trivy.
Features — confirmed present and functionally wired, not placeholders	• Grew the Jenkinsfile from the teammate's 103--106 lines to 277 lines: added parallel npm-audit + ESLint stages, a parallel Trivy container-scan + Docker smoke-test stage, multi-platform (amd64/arm64) Docker builds via buildx, and a conditional Deploy stage wrapped in try/catch so a missing deploy target doesn't fail the whole pipeline. • New AWS Secrets Manager module for the Jenkins admin password, a dedicated IAM module scoped to the specific secret ARN (not a wildcard), and a new VPC-endpoints module (S3 gateway + Secrets Manager interface endpoint) so instances reach these services without the public internet.
Bug Fixes — both verified via direct diff	• Fixed a real test-breaking issue: app.js previously called app.listen() unconditionally at module load, so importing it in the test file also opened a real port. Fixed with an if (require.main === module) guard. • Fixed an insecure SSH pattern: the Deploy stage's sshagent (SSH-agent-forwarding) block was removed entirely and replaced with withCredentials([sshUserPrivateKey(...)]) — confirmed no sshagent reference remains anywhere in the current Jenkinsfile.

Notable / High-Impact Contributions	• Pipeline hardening, verified end-to-end: the parallel security stages and multi-platform builds are real, wired, functioning stages in the current file, not cosmetic additions — the single clearest evidence-backed piece of engineering growth in the repo. • A deliberate, traceable Terraform hardening sequence (Secrets Manager, scoped IAM, tightened egress, VPC endpoints), all same-day, independently confirmed coherent by reading the module wiring.
Caveat	The repo currently has a real SSH private key (terraform/cicd-pipeline-keypair.pem) and Terraform state files committed to git — a genuine hygiene issue sitting oddly next to the security-hardening work done elsewhere in the same pipeline.

D.1 3tier-project	
GitHub	https://github.com/Benjamin-yankey/3tier-project
Timeframe	2025-12-22 to 2026-01-27, 11 commits, solo (100% of repo history).
Features — re-verified directly against current .tf files	• RDS hardening confirmed: multi_az = true, storage_encrypted = true, iam_database_authentication_enabled = true, publicly_accessible = false. • DB security group confirmed: only allows port 3306 ingress from the app security group — no 0.0.0.0/0 rule remains on the DB tier. • Secrets Manager confirmed: a random_password resource is generated and stored, wired into both database and compute modules, with db_password marked sensitive = true; no hardcoded password string exists in any current .tf file.
But History Contradicts the "Secrets Fully Secured" Narrative	Commit b9e2890 ("Bad Gateway") committed a full terraform.tfstate.*.backup file — Terraform state stores resolved attributes in plaintext, so this backup contained the real, at-the-time DB password. The same commit also added an SSH private key (3tier-key.pem), removed one commit later — but neither commit was purged from history, so both secrets remain permanently recoverable via git show even though HEAD is clean.
SSH/Bastion Lockdown — Partially Confirmed, With a Real Caveat	The mechanism genuinely exists (the bastion SG scopes SSH to an admin_cidr variable instead of a hardcoded 0.0.0.0/0), but variables.tf defaults admin_cidr to "0.0.0.0/0" with only a # TODO comment, and the committed terraform.tfvars never overrides it — so as actually configured, SSH to the bastion is still open to the entire internet by default.

Additional Findings	• A superfluous port-443 inbound rule on the app-tier SG, labeled for SSM Session Manager, reflects a misunderstanding of how SSM works (it needs no inbound listener) and reopens a public port the rest of the hardening was trying to close. • 942 files (6.2MB) of an orphaned node_modules/ tree are committed with no corresponding package.json anywhere in history — an accidental commit, not intentional dead code, since the actual to-do app is external and runs its own npm install on the EC2 instance. • The README claims MySQL 8.0 or PostgreSQL 14 support; the code has no Postgres path at all — a doc/code mismatch.
Bug Fixes	Diagnosed and fixed a live "Bad Gateway" incident (b9e2890): switched EC2 AMI, rewrote the bootstrap script, added a DB-connectivity retry loop and a bounded health-check gate before marking the instance healthy — confirmed still present and functional in the current user-data script.
Notable / High-Impact Contributions	• The security-hardening pass is real, working infrastructure change, independently confirmed against current code — the most substantive technical work in the repo. • The Bad Gateway incident response is genuine debugging across the OS/ bootstrap/dependency stack.
Caveat	The application deployed by this infrastructure (a to-do app) is entirely external — cloned from a third-party GitHub repo at boot time. No application source code exists in this repository; only the deployment/infrastructure logic is this author's own verifiable work.

D.2 aws-automation-project	
GitHub	https://github.com/Benjamin-yankey/aws-automation-project
Timeframe	2025-12-12 to 2026-01-16, 14 commits, solo.
Features — re-verified as genuinely shared/functional, not just claimed	• c14e8dd, 1515573, d83b1e0 — state_manager.sh: a real S3-backed JSON state tracker with genuine format-migration logic, confirmed by diffing the original 190-line version against the current 514-line version. • ```\${=html} <!-- --> ``` lib/common_utils.sh (458 lines) confirmed genuinely sourced and called (not dead code) by all four top-level scripts and state_manager.sh. • Dry-run enforcement confirmed real: every destructive AWS call in the scripts is preceded by a guard that returns early, not just a printed message alongside a still-firing real call. • Security defaults confirmed enforced in logic, not just documented: encryption-on and public-access-off are real put-bucket-encryption/put-public-access-block calls gated on boolean flags defaulting to safe values.
Bug Fixes	Fixed a macOS-specific mktemp incompatibility and a dry-run guard that was over-blocking state-file existence checks, both confirmed via direct diff.

Notable / High-Impact Contributions	• The state management system is the most technically substantial original design in the repo — a working, self-migrating state tracker built in pure Bash/jq. • The shared-utility-library extraction is a genuine, verified refactor, not code left unused after extraction.
--	---

D.3 DataPulse (solo scaffold)	
GitHub	https://github.com/Benjamin-yankey/DataPulse
Timeframe	2026-03-05, single day, 8 commits, solo (distinct from the team-based DataPulse_Team8).
What's Genuinely Complete	Auth (register/login, real JWTs) and dataset upload (real pandas CSV/JSON parsing + persistence) are the only two feature areas fully wired to real logic and covered by passing tests — independently re-verified by running the actual test suite in a clean, correctly-pinned virtualenv (4/4 backend tests pass).
What's Still Stub Code — Re-Verified Against Current Code	• <code>scoring_service.calculate_quality_score()</code> unconditionally returns a hardcoded 0.0 with a TODO describing the intended weighted-scoring algorithm that was never written. • <code>validation_engine.py</code>: of 5 rule types, only NOT_NULL does real work; the other 4 (DATA_TYPE, RANGE, UNIQUE, REGEX) are stubs returning "not yet implemented." • <code>report_service.py</code> is a full stub, and the checks/reports routers never even reach it — they raise <code>HTTPException(501)</code> immediately.
Bug Fixes	A <code>bcrypt</code> UTF-8 byte-truncation bug (naive character-slicing miscounting multi-byte input against <code>bcrypt</code> 's 72-byte limit) was identified and properly fixed with a dedicated encode-then-truncate helper — independently re-verified as correct by running the real test suite with exact pinned dependency versions.
CI/CD — a New Finding on Re-Verification	The lint job is not durably reproducible: <code>ruff</code> is installed with no version pin and no repo-level ruleset config, so re-running <code>ruff</code> check today (a newer <code>ruff</code> version) surfaces 76 findings that weren't failing on the original commit date. The <code>pytest</code> job's pass/fail claim held up when re-tested with pinned dependencies; the lint job's did not hold up under a fresh, unpinned install. A second, nested CI config (<code>devops/.github/workflows/ci.yml</code>) is also dead — GitHub Actions never discovers workflows outside the repo-root <code>.github/workflows/</code> path.
Notable / High-Impact Contributions	• The <code>bcrypt</code> fix is the one piece of genuine debugging judgment in the history, independently re-confirmed as correct and functional. • The auth + upload vertical slice is real, tested, working FastAPI/SQLAlchemy/pandas integration — the rest of the API surface is scaffolding.
Caveat	This should be described as "a working scaffold with one complete feature slice and a CI pipeline whose lint gate isn't reproducible," not as "a built data quality platform."

D.4 IaC	
GitHub	https://github.com/Benjamin-yankey/IaC
Timeframe	2026-01-28, 2 commits ~22 minutes apart, solo.
Stack	Terraform (AWS provider ~6.0), a modular VPC/security/keypair/EC2 layout with an S3 remote-state backend.
Overview	• Features — all four modules read in full and confirmed complete, not skeletal • VPC module (public subnet, IGW, route table), security module (SSH from one admin IP, HTTP from anywhere), keypair module (a genuine 4096-bit RSA key generated via <code>tls_private_key</code> and registered in AWS), and an EC2 module wired to all three, with real CIDR/IP input validation blocks. • Remote state backend confirmed to use Terraform's native S3 lockfile locking (<code>use_lockfile = true</code>) rather than the older DynamoDB lock-table pattern — a current best practice, correctly configured with encryption on.
Documentation Discrepancy	The README's "Author" section names a different person ("Asher Yram Tetteh-Abotsi") than the git history shows. Both commits are unambiguously authored and committed by Benjamin Kofi Yankey, and the byline was present from the very first commit and never edited. The most plausible, honest read is stale boilerplate carried over from a shared training template, not a false authorship claim — but it's a real inaccuracy worth correcting at the source. Separately, a screenshot and one architecture-doc line reference "optional DynamoDB locking" that isn't actually implemented — leftover artifact from an earlier draft approach, not a functional defect.
Notable / High-Impact Contributions	• A complete, coherently-wired four-module Terraform stack with real input validation and remote state, delivered in a single commit — unusually complete for a first commit. • Correct use of current S3 native state locking rather than the older DynamoDB pattern many tutorials still default to.

D.5 OpenStack	
GitHub	https://github.com/Benjamin-yankey/OpenStack
Timeframe	2025-12-23, 1 commit, solo. Despite the folder name, unrelated to the OpenStack platform — a small AWS serverless demo.
Stack	Terraform (AWS + archive provider), Python 3.11 Lambda handler.
Overview	• Features — all three modules read in full and confirmed complete • An API Gateway REST API with proxy integration in front of a Python Lambda, plus a dedicated IAM module for the Lambda execution role — a genuine, deployable minimal serverless pattern confirmed end-to-end from HTTP request through routing to the handler's response shape.

Tech Debt Found	• The IAM module attaches the AWS-managed basic-execution policy and also adds a redundant custom logging policy that duplicates it, scoped to a wildcard log-group ARN rather than the specific one — not tightened to least-privilege despite the README's claim, though the blast radius is small since no other AWS access is granted. • An orphaned bootstrap-web.sh script (installs Apache) is not referenced by any Terraform resource and doesn't match the Lambda/API-Gateway architecture actually deployed — dead code, likely copied in from a different exercise.
Notable / High-Impact Contributions	The modular api_gateway/iam/lambda split is real and correctly wired (verified cross-module output references), a legitimate demonstration of production-style Terraform organization at small scale.

D.6 terraform-ansible-webapp	
GitHub	https://github.com/Benjamin-yankey/terraform-ansible-webapp
Timeframe	2026-01-20 to 2026-01-29, 8 commits, solo.
Stack	Terraform (EC2/SG/elastic IP), Ansible (Jinja2 templates, dynamic inventory), Flask+SQLAlchemy API, React frontend, Nginx, systemd, firewalld, SELinux, auditd, Open Policy Agent (Rego).
Overview	• Features — a real, working task-manager app, confirmed by reading the code directly • Full CRUD Flask API (tasks, categories, stats, health) backed by SQLAlchemy models with filtering/search/validation, and a real React client (create/edit/delete/toggle-complete, category/priority filters, live search, a stats dashboard) — not a stub or placeholder app.
Genuine Security Hardening — verified directly in the Ansible tasks	• firewalld and SELinux concretely configured (specific ports, specific booleans, specific SELinux context), though most of these tasks use ignore_errors: true, which silently swallows failures rather than surfacing them. • Real systemd sandboxing on the backend service (NoNewPrivileges, PrivateTmp, ProtectSystem=strict, restricted ReadWritePaths) and a genuine auditd/rsyslog logging playbook watching passwd/shadow/sudoers/sshd_config/systemd/kernel-module changes.
Real But Disconnected Security Tooling	A working bash threat-detection script (checks failed SSH logins, suspicious processes, sudo-escalation patterns, unexpected open ports, CPU/memory thresholds) and seven substantive OPA/Rego policy rules both exist and function standalone — but neither is actually wired into the deployment: nothing schedules the script via cron/systemd, and nothing in the repo invokes OPA/conftest against the policies. The project's own docs candidly admit the script isn't scheduled yet.
CI Add-Then-Remove	A fairly complete "security-gates" GitHub Actions pipeline (ansible-lint, terraform validate, shellcheck, bandit, pylint, gitleaks, pre-commit) was built and then deliberately deleted two days later during cleanup — confirmed no .github directory exists in the repo today. Presented honestly as an experiment that was tried and rolled back, not a capability the finished project retains.

Notable / High-Impact Contributions	The security-hardening commit is genuine security engineering effort — firewall/SELinux/systemd sandboxing/auditd all concretely configured, not just packages installed and left alone.
Caveat	The security code is real and non-trivial, but the security system — end to end, automatically enforced — is not fully assembled: the threat-detection script and the Rego policies are both currently disconnected from anything that runs them.

D.7 terraform-aws-infrastructure

GitHub	https://github.com/Benjamin-yankey/terraform-aws-infrastructure
Timeframe	2025-12-24 to 2026-01-13, 3 commits, solo.
Stack	Terraform (VPC/EC2/S3/DynamoDB remote state).
Features	A VPC with a public subnet, an EC2 web server running Apache (confirmed installed via user_data, not a separate provisioner) with an index page showing the instance ID/AZ, and an S3+DynamoDB remote-state backend, plus a backend.sh bootstrap script that correctly creates the bucket/lock table only if they don't already exist.
Minor Doc/Code Mismatches Found	• backend.tf's own comments describe a two-step "deploy with the block commented out, then uncomment" process, but the block is already active/uncommented in the current file — the instructions no longer match the file's own contents. • variables.tf defaults to t3.micro while the README documents t2.micro throughout — a minor doc/code mismatch.
Notable / High-Impact Contributions	The entire working infrastructure — network, compute, and remote-state backend — was delivered in one comprehensive first commit (~950 lines), with a later commit correctly fixing the state-locking mechanism to match the DynamoDB table actually provisioned.

D.8 yaml-config-lab

GitHub	https://github.com/Benjamin-yankey/yaml-config-lab
Timeframe	2026-01-05 to 2026-01-29, 3 commits, solo. A YAML/JSON configuration-management training lab.
What Was Genuinely Attempted	app-config.yaml was properly rewritten from a hardcoded plaintext password to environment-variable interpolation, and a real Flask app was written to actually consume that config (not just decorate the README) — a genuine attempt to wire a real consumer to the security fix.

What Doesn't Actually Work — Confirmed by Direct Execution	• The secret-externalization is incomplete: a sibling app-config.json file still has the plaintext password from the first commit and was never touched again. • Several scripts are non-functional as committed: test-config.sh, validate-config.sh, and config-schema.json were all saved with literal n/t characters instead of real newlines/tabs. Running them directly confirmed both shell scripts execute silently and exit 0, performing none of their described checks. • The demo app crashes on startup: the config relies on \${VAR:-default} bash-style defaulting, but the only substitution tool actually used (envsubst) doesn't support that syntax — confirmed by running the app's own startup path, which throws a ValueError before the server can bind a port. • A Kubernetes manifest template is similarly corrupted (one line, literal n characters, invalid YAML) and is never referenced by any script in the repo. The Makefile itself fails immediately on make test/make validate with a syntax error.
Notable / High-Impact Contributions	The intent — recognizing a plaintext password as a security problem and building a real (if ultimately non-functional) pipeline to verify its removal — is a reasonable learning objective. But this should be reported plainly as an ambitious scope that outran what was actually tested before committing, not as a working security-hardening deliverable.

E.1 git-lab	
GitHub	https://github.com/Benjamin-yankey/git-lab
Timeframe	2025-12-01 to 2025-12-10, 5 commits, solo. No application code — a self-contained Git workflow training exercise.
Verified as Genuinely Executed, Not Narrated	• A real feature-update branch exists (confirmed an ancestor of main), and the live reflog contains 8 real entries including "merge feature-update: Fast-forward" — a fast-forward merge was actually run, and the reflog-inspection task was actually performed, not just written about. • The README's documented commit messages exactly match the real commit messages in git log, and all three referenced screenshots are physically present in the repo — confirming the README describes what was actually done rather than a copy-pasted template.
Notable / High-Impact Contributions	The branch/merge/reflog sequence is the strongest evidence of real practice here: genuine hands-on execution of the core Git workflow (branch, fast-forward merge, GitHub push, reflog inspection), not a simulated exercise.

E.2 devops_challenge	
GitHub	https://github.com/Benjamin-yankey/devops_challenge
Timeframe	2025-12-01 to 2025-12-09, 6 commits, solo. Bash only.

Verified by Direct Execution	• Idempotency fix is real but incomplete: removing a destructive <code>rm -rf</code> did stop the script from wiping content on a second run, but running the script twice in a clean test directory still fails outright with a Permission denied error on a file left read-only (444) by the first run — full idempotency was never actually achieved. • The "shebang bug fix" commit did not fix a bug — it introduced one, still present at HEAD: the commit collapsed a correct multi-line <code>echo</code> into a single line that combines the shebang and the command; running the generated scripts/<code>backup.sh</code> directly fails with "cannot execute binary file," and running it via <code>bash</code> produces no output at all, confirmed via <code>od -c</code> on the actual generated file. • The tree-command fallback is a legitimate, verified fix: given <code>set -e</code> was active by that point, an unguarded <code>tree</code> call would have aborted the whole script on any machine without <code>tree</code> installed.
Notable / High-Impact Contributions	• Recognizing and partially fixing the destructive <code>rm -rf</code> is real safety thinking, even though full idempotency wasn't reached. • The tree fallback is a legitimate portability fix.
Caveat	Hands-on re-testing found the project stopped mid-fix rather than being fully polished: the generated backup script is currently broken, and the setup script is not actually safe to re-run.

E.3 Data Structures and Algorithms	
GitHub	https://github.com/Benjamin-yankey/Data-Structures-and-Algorithms
Timeframe	2025-12-12 to 2025-12-22, 9 commits, solo. Python 3, standard library only.
Features — verified by manual tracing and execution	• Recursion/Backtracking (255 lines): three independent implementations of "generate all well-formed parentheses" (recursive, DP, explicit-stack), a validator, and a self-written test suite checked against known Catalan numbers — manually traced for $n=2$ and confirmed correct. • Kadane's algorithm and array rotation: both re-traced and confirmed correct, each with dedicated test functions covering edge cases (all-negative input, k larger than array length).
Correction From Re-Verification	The earlier "quicksort bug fix" framing was inaccurate. On inspection, the first commit's file (<code>Quick_Sort.py</code>) does not implement quicksort at all — it's a quickselect-based solver for an unrelated "minimum subarray sum difference" problem, mislabeled. The second commit's file (<code>QuickSortAlgorithm.py</code>) is a real, correct quicksort, manually traced against a sample array and confirmed to sort correctly. The mislabeled first file was never removed and still sits in the repo — a stray-file issue caught late, not a cleanly resolved bug.
Notable / High-Impact Contributions	• The recursion/backtracking exercise is the clear standout — real engagement with why the algorithm works, independently re-verified by hand-tracing, not just accepting the code's own claims. • The quicksort mislabeling episode is valuable less for the code and more as visible (if incomplete) evidence of self-review.

E.4 bashScripting	
GitHub	https://github.com/Benjamin-yankey/bashScripting
Timeframe	2025-12-22, 1 commit. 19 scripts, all executed and verified.
Verified by Direct Execution	• A real, confirmed bug in password.sh: correct_password = "1234567890" has spaces around =, invalid Bash syntax. Bash silently leaves the variable unset, so pressing Enter (empty input) at the password prompt is accepted as valid — confirmed by running the script. • redirectCal.sh "works" (echoes 20) for the wrong reason: the input file contains the literal line num=10 rather than 10, so the arithmetic is actually evaluating an assignment expression, not cleanly doubling a read value. • Two scripts (bothterminalfile.sh, OutputRedirection.sh) wrote their output into hello.sh instead of a dedicated scratch file, so the committed hello.sh no longer contains the original "Hello World" script — confirmed against the actual committed git blob.
Notable / High-Impact Contributions	Introductory practice work — nothing rises to high-impact. The password.sh bug is a good, concrete lesson in why Bash's no-spaces-around-= rule matters, since the failure mode is a silent security hole rather than a crash.

E.5 app	
GitHub	https://github.com/Benjamin-yankey/app
Timeframe	2025-12-18 to 2026-01-22, 3 commits, solo. Node.js/Express/MySQL2 against AWS RDS.
Features	A working contact-form flow (form → Express routes → MySQL/RDS persistence with connection-retry logic), later a /health endpoint and CORS for a load balancer. SQL-injection check: all queries confirmed parameterized — no injection risk found.
New Security Findings on Re-Verification	• .env is committed to git since the first commit, containing a real-looking AWS RDS hostname, an admin username, and a plaintext password — permanently visible in git history even if since rotated. • Stored/reflected XSS risk: user-submitted name/email/message values are inserted directly into HTML template literals with no escaping, so a submission containing script tags would execute in another viewer's browser. • CORS is fully permissive (allow-all origin/methods including DELETE), loose for an app exposing a delete endpoint, though impact is limited since there's no authentication to bypass in the first place.
Tech Debt (confirmed)	An unused multer dependency and an empty, untracked uploads/ directory; a hardcoded PORT = 3000 that never actually reads process.env.PORT despite dotenv being loaded; the entire server lives in a file literally named Node.js rather than server.js/app.js.

Notable / High-Impact Contributions	• Too small a sample (3 commits, one file) to claim scope growth. The initial commit is the most defensible piece of evidence: a working end-to-end flow with real connection-retry handling — reasonable basic backend/cloud-integration competency, alongside real, newly-identified rough edges (committed credentials, XSS) worth naming plainly. • F. No Git History, But Genuine Technical Content Verified by Reading the Files • These four folders were never committed to git, so none of them carry dates, authorship, or an iteration history — but unlike the truly empty/trivial folders in Section H, direct inspection of the files (and, in three cases, an actually-applied Terraform state file) confirms real, substantive technical work exists. They're presented here as verified content without a timeline, not as dated contributions, and are not used to support any specific claim of growth or scope in Part 1.
--	---

F.1 try	
GitHub Overview	https://github.com/Benjamin-yankey/try A Terraform config provisioning an AWS RDS MySQL instance, paired with an Express "contact form" app that reads/writes to it. Confirmed actually deployed, not just written: a populated Terraform state backup shows a real, live-looking RDS endpoint, and the app's own .env points at that exact database. The database was later torn down (the live state file is now empty). Real, working, honest flaw: the DB password is hardcoded in plaintext in both main.tf and .env, and the instance was configured publicly_accessible = true with MySQL's port open to 0.0.0.0/0. File modification times cluster around Dec 18, 2025, and the code closely resembles an earlier, uncommitted draft of the separate, git-tracked app repository (identical dependency set, same distinctively-named Node.js entry file) — plausibly a precursor to it, though this is an inference from file layout, not proof.

F.2 terraform	
GitHub Overview	https://github.com/Benjamin-yankey/terraform A materially more mature, modular Terraform environment than any other IaC exercise in this portfolio — six modules (vpc, security, secrets, iam, compute, database) implementing a layered bastion → frontend → backend → database security-group chain, Secrets Manager + random_password for DB credentials, and a scoped (not wildcard) IAM policy. Confirmed actually deployed and later destroyed: a state-file backup lists 39 real managed resources matching the module code closely; the current state file is empty, indicating a full destroy cycle. This is the strongest piece of evidence in the folder — the infrastructure was validated by actually running it in AWS, not just written and left untested. No README exists, and a copy-paste artifact left both "frontend_public_ip" and "backend_public_ip" outputs pointing at the same bastion IP.

F.3 Containerized	
GitHub	https://github.com/Benjamin-yankey/Containerized
Overview	A three-tier Docker Compose app (Nginx → Express → Postgres) with genuinely correct container-security fundamentals: a real multi-stage Dockerfile build, a dedicated non-root user, and a container-level HEALTHCHECK actually wired to the app's own /health route — plus real Docker network segmentation (the database service has no host-published port and only joins the backend network, matching the architecture docs' isolation claim exactly, not just diagrammed). Confirmed an actual AWS deployment was attempted: an applied Terraform state file contains a live EC2 instance ID and public IP, matched by the same hardcoded IP inside a troubleshooting script — the author hit a real IAM permission/connectivity failure and documented a fallback to local Docker Compose. Minor doc-drift found: the README documents port 8080 while the actual Compose file maps 8081, and the Ansible playbooks describe a more elaborate multi-host topology than what the Compose file actually runs.

F.4 Orchestrator (quote-api)	
GitHub	https://github.com/Benjamin-yankey/Orchestrator
Overview	• A side-by-side lab comparing AWS ECS Fargate against Kubernetes, deploying the identical trivial "Quote of the Day" API both ways to observe the same self-healing and autoscaling behavior under each. Both orchestrator configurations are genuinely complete and symmetric — real ECS task definitions with health checks and Application Auto Scaling at a 50% CPU target, and a real Kubernetes Deployment/Service/HorizontalPodAutoscaler set at the same threshold — plus a detailed, technically accurate runbook whose troubleshooting section (specific symptoms like ImagePullBackOff or HPA showing <unknown>) reads as though it was written after actually hitting these issues. • <i>Caveat: there is no evidence file anywhere (no logs, screenshots, or captured output) proving this was ever actually run end-to-end. The ECS task definition still contains unsubstituted <ACCOUNT_ID>/<REGION> placeholders, and every "you should see" line in the runbook describes expected output rather than pasted real output. This should be read as a complete, coherent design/runbook, not confirmed proof of a working, tested dual deployment.</i>

G. Repositories Excluded From Evidentiary Claims

Listed only for completeness — either empty, unimplemented, trivial, or not attributable to this author. None of these are cited as evidence anywhere in Part 1.

R e p o s i t o r y	G o i t s H u b	W h y it s e x c l u d e d
		a m i n i m al , t u t o ri al - st y le si n g le - E C 2 - i n st a n c e T e rr a f o

Why it's the best place to buy a house

at the moment, the market is very hot and it's difficult to find a house that meets your requirements. The best way to find a house is to use a real estate agent who can help you find a house that meets your requirements.

at the moment, the market is very hot and it's difficult to find a house that meets your requirements. The best way to find a house is to use a real estate agent who can help you find a house that meets your requirements.

R e p o s i t o r y	G o i t s H u b	W h y i t s e x c l u d e d
		p a c k a g e. js o n d e s c r i b e s a n a m b i t i o u s p l a n n e d m u l t i- p r

Why it's excluded

just a bare representation of the package. js on the declared a ring on the dependence (l

Why it's excluded

https://github.com/imviele/njtalomyeinmyesnmstbnkyeflyoladventeronaanlibela

Why it's excluded

all 4 comments it's below on getting to a team matter, Max well
[http://sigithub.com/](#) we see his idea was implemented by R.

